

Informe de la Pràctica: Containers amb Docker

Nerea de la Torre, 1669013 i Adrián Prego, 1672251

Introducció

En aquesta pràctica ens centrem en l'ús de contenidors Docker com a eina per empaquetar, distribuir i executar aplicacions de manera eficient i portable. Amb d'exercicis pràctics, treballarem els conceptes fonamentals de Docker, com la creació d'imatges personalitzades mitjançant Dockerfiles i la gestió del cicle de vida dels contenidors. També s'utilitzarà AWS Cloud9 com a entorn de desenvolupament integrat al núvol per facilitar el treball amb contenidors.

Objectius

L'objectiu principal d'aquest laboratori és adquirir competències pràctiques en l'ús de Docker per:

- Crear imatges personalitzades definides amb **Dockerfiles**
- Gestionar el cicle de vida complet dels contenidors (creació, execució, monitoratge i eliminació)
- Desplegar aplicacions en contenidors aïllats, incloent aplicacions **Python**
- Comprendre l'estructura de capes que forma una imatge Docker
- Utilitzar **AWS Cloud9** com a entorn de desenvolupament per treballar amb contenidors

Entorn de desenvolupament i lliurament

1. Accés a AWS Academy i Cloud9

Ens hem connectat a AWS Academy, iniciat el laboratori i accedit a la consola de gestió d'AWS.

2. Creació de l'entorn de desenvolupament Cloud9

Un cop dins la consola, hem cercat el servei **Cloud9** i hem creat un nou entorn seleccionant l'opció de connexió *Secure Shell (SSH)*. Hem proporcionat el nom *Pràctica 5* a l'entorn i l'hem creat.

3. Preparació del codi

Al terminal de Cloud9, hem descarregat l'arxiu **code.zip** amb el següent comandament:

```
1 wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACCDEV-2/lab-06-containers/code.zip -P /home/ec2-user/environment
2
3 cd /home/ec2-user/environment unzip code.zip
```

Després hem executat el script de preparació amb:

```
1 chmod +x ./resources/setup.sh && ./resources/setup.sh
```

4. Verificació de l'entorn

Hem comprovat la versió de la CLI d'AWS:

```
1 aws --version
```

I hem comprovat si el SDK per a Python (**boto3**) està instal·lat:

```
1 pip show boto3
```

Exercici 1: Configuració de Dockerfile

En aquesta primera tasca, hem utilitzat AWS Cloud9 per crear i executar contenidors Docker dins la mateixa instància EC2 que allotja l'IDE. Això ens ha permès generar la imatge i posar en marxa el contenidor de manera integrada i eficient.

Hem seguit aquesta estratègia realitzant els passos següents:

Creació del projecte Docker

a) Directori i script Python

Hem creat un nou directori per a l'aplicació:

```
1 mkdir containers
2 cd containers
```

A dins hem carregat l'script **showtime.py**:

```
1 import time
2 import datetime
3
4 print("Hello Docker")
5 for i in range(100):
6     current_time = datetime.datetime.now()
7     print("Current time:", current_time)
8     time.sleep(5)
```

b) Dockerfile

Hem creat un fitxer **Dockerfile** amb el contingut següent:

```
1 FROM node:11-alpine
2 RUN apk add --update --no-cache python3
3 RUN mkdir -p /opt/app
4 COPY showtime.py /opt/app
5 RUN chmod 744 /opt/app/showtime.py
6 WORKDIR /opt/app
7 ENTRYPOINT ["python3", "-u", "/opt/app/showtime.py"]
```

Aquest fitxer conté les instruccions necessàries per construir una imatge de contenidor a partir d'una base Alpine amb Node i Python. L'script **showtime.py** s'executarà cada 5 segons en bucle infinit.

6. Construcció i execució del contenidor

Per construir la imatge Docker amb etiqueta **showtime:v1**, hem executat:

```
1 docker build --tag showtime:v1 /home/ec2-user/environment/containers
```

Un cop construïda, hem llançat el contenidor:

```
1 docker run --name time -it showtime:v1
```

Per aturar l'execució hem premut **Ctrl+C**. Posteriorment hem eliminat el contenidor:

```
1 docker rm time
```

I hem comprovat els contenidors existents:

```
1 docker ps -a
```

7. Execució en segon pla

Per executar el contenidor en segon pla i eliminar-lo automàticament fem:

```
1 docker run --rm -d --name time showtime:v1
```

Hem comprovat que el contenidor s'executa amb:

```
1 docker container ls
2 docker logs time
3 docker ps -a
```

Finalment, l'hem aturat amb:

```
1 docker stop time
```

Pregunta

Quines són les capes d'aquest contenidor que heu creat? Fes una llista explicant cadascuna d'elles.

Cada instrucció del Dockerfile genera una capa nova a la imatge. A continuació detallem cadascuna:

- **FROM node:11-alpine:** Capa base de la imatge, una versió mínima de Linux amb Node.js.
- **RUN apk add --update --no-cache python3:** Capa que instal·la la Python3 a la imatge.
- **RUN mkdir -p /opt/app:** Crea el directori on s'ubicarà l'aplicació.
- **COPY showtime.py /opt/app:** Capa que afegeix el nostre script Python dins del contenidor.
- **RUN chmod 744 /opt/app/showtime.py:** Afegeix permisos d'execució a l'script.
- **WORKDIR /opt/app:** Defineix el directori de treball.
- **ENTRYPOINT ["python3", "-u", "/opt/app/showtime.py"]:** Especifica l'ordre que s'executarà quan el contenidor arrenqui.

Aquestes capes permeten que el contenidor executi un script que imprimeix l'hora actual cada 5 segons, i formen part de la imatge final que hem creat.

Exercici 2: Aplicacions web de contenidors

En aquest exercici, s'havia de treballar amb un contenidor Docker orientat a desplegar una aplicació web, que era el següent Dockerfile:

```
1 FROM node:11-alpine
2 RUN mkdir -p /usr/src/app
3 WORKDIR /usr/src/app
4 COPY . .
5 RUN npm install
6 EXPOSE 3000
7 CMD ["npm", "run", "start"]
```

Aquest fitxer defineix les instruccions necessàries perquè Docker construeixi una imatge per a una aplicació web escrita en Node.js. A continuació, detallem

les capes que es generen durant la creació d'aquesta imatge:

- **FROM node:11-alpine:** Aquesta és la capa base de la imatge, que utilitza una versió lleugera del sistema operatiu Alpine Linux amb Node.js 11 pre-instal·lat. És la base mínima sobre la qual es construeixen les capes següents.
- **RUN mkdir -p /usr/src/app:** Afegeix una nova capa on es crea el directori de treball per a l'aplicació web.
- **WORKDIR /usr/src/app:** Defineix el directori de treball on s'executaran les ordres següents. Aquesta capa no afegeix fitxers, però estableix el context d'execució.
- **COPY . .:** Crea una capa que copia tot el contingut del directori local al directori de treball del contenidor. És una de les capes més importants, ja que incorpora el codi font i fitxers de configuració.
- **RUN npm install:** Aquesta capa executa la instal·lació de totes les dependències del projecte definides a `package.json`, creant així la carpeta `node_modules`.
- **EXPOSE 3000:** Indica que el contenidor escoltarà al port 3000. Aquesta capa és informativa i serveix perquè altres serveis o usuaris sàpiguen quin port utilitza l'aplicació.
- **CMD ["npm", "run", "start"]:** Defineix la comanda per defecte que s'executarà quan el contenidor es posi en marxa. Aquesta comanda llança l'aplicació web.

Aquest Dockerfile ens permet empaquetar una aplicació web de Node.js en una imatge reutilitzable i portable. Un cop creada la imatge, podem executar-la en qualsevol entorn que tingui Docker instal·lat, garantint que el comportament de l'aplicació sigui coherent i predictable.

Exercici 3: Contenedor per a l'script api_example.py

En aquest exercici dissenyem i construïm una imatge Docker que executi un script Python per consultar les tres notícies més recents del tema "World" als EUA i mostrar-ne títol i enllaç. Es fa mitjançant el paquet `gnewsclient`. A continuació detallem els passos i les instruccions utilitzades.

1. Script api_example.py

El primer pas ha estat carregar l'script `api_example.py` dins del directori `containers`. Aquest fitxer conté el codi Python que utilitza la biblioteca `gnewsclient` per obtenir i mostrar les últimes notícies. A continuació mostrem el seu contingut:

```
1 from gnewsclient import gnewsclient
2
3 client = gnewsclient.NewsClient(
4     language='english',
5     location='United States (English)',
6     topic='World',
7     max_results=3
8 )
9
10 news_list = client.get_news()
11 for item in news_list:
12     print("Title : ", item['title'])
```

```

13 print("Link : ", item['link'])
14 print("")

```

2. Dockerfile

A continuació hem creat un fitxer Dockerfile amb el contingut següent (es mostren els passos):

```

1 # 1. Base: Python 3.8 sobre Alpine
2 FROM python:3.8-alpine
3
4 # 2. Instal·lem dependències de compilació (si
5   cal) i el paquet gnewsclient
6 RUN apk add --no-cache build-base \
7   && pip install --no-cache-dir gnewsclient
8
9 # 3. Creem un directori per al codi
10 RUN mkdir -p /opt/app
11
12 # 4. Copiem l'script dins la imatge
13 COPY api_example.py /opt/app/
14
15 # 5. Marquem el directori de treball
16 WORKDIR /opt/app
17
18 # 6. Punt d'entrada per executar l'script quan
19   arrenqui el contenidor
20 ENTRYPOINT [ "python", "api_example.py" ]

```

Capes de la imatge Docker

Les capes generades per les instruccions del Dockerfile són:

1. **FROM python:3.8-alpine:** imatge base amb Python 3.8 i pip.
2. **RUN apk add && pip install:** instal·lació de build-base i gnewsclient.
3. **WORKDIR /opt/app:** definició del directori de treball.
4. **COPY api_example.py :** còpia de l'script dins la imatge.
5. **ENTRYPOINT:** definició de la comanda d'arrencada.

3. Operacions per copiar i executar el codi Python

- **WORKDIR /opt/app:** defineix el directori on s'executaran les comandes posteriors.
- **COPY api_example.py :** còpia l'arxiu api_example.py dins del contenidor.
- **ENTRYPOINT ["python", "api_example.py"]:** estableix la comanda que s'executarà en arrencar el contenidor.

4. Creació, execució i monitorització del contenidor

A continuació, detallem el procés per construir, executar i monitorar el contenidor Docker, utilitzant les comandes i el procés explicat a l'exercici 1:

Primer ens assegurem d'estar al directori containers, i creem el fitxer Dockerfile i l'omplim amb el contingut mostrat en la secció anterior.

```
1 touch Dockerfile
```

Construcció de la imatge: Seguim en el directori containers, i utilitzem l'ordre següent per generar la imatge Docker anomenada **api-example:v1**:

```
1 docker build -t api-example:v1 .
```

```

voclabs:~/environment/containers $ touch Dockerfile
voclabs:~/environment/containers $ docker build -t api-example:v1 .
[+] Building 15.9s (10/10) FINISHED                                docker:default
-> [internal] load build definition from Dockerfile                0.0s
-> => transferring dockerfile: 615B                                0.0s
-> [internal] load metadata for docker.io/library/python:3.8-alpine 0.4s
-> [internal] load .dockerignore                                   0.0s
-> => transferring context: 2B                                       0.0s
-> [1/5] FROM docker.io/library/python:3.8-alpine@sha256:3d93b1f77efce339aa77db7266 1.7s
-> => resolve docker.io/library/python:3.8-alpine@sha256:3d93b1f77efce339aa77db7266 0.0s
-> => sha256:9f956f4ecf0013796bda95d2ed2181cf22e23036a1daa1010 1.73kB / 1.73kB 0.0s
-> => sha256:c89247b02f92df046086379ef1b787431169ac105b8e9cbcd2 4.95kB / 4.95kB 0.0s
-> => sha256:43c4264eed91be63b206e17d93e75256a697070ce43c5e8f0379 3.62kB / 3.62kB 0.1s
-> => sha256:bdd029b7f2dc0cbe5a1fb25d1006dc58892c476b385af7243 455.12kB / 455.12kB 0.1s
-> => sha256:2fe9ed713579c280eb3576e25f4604247ceb5d36482a5bb477de 14.22kB / 14.22kB 0.4s
-> => sha256:3d93b1f77efce339aa77db726656872517b0d67837989aa7cab3 10.29kB / 10.29kB 0.0s
-> => extracting sha256:43c4264eed91be63b206e17d93e75256a697070ce43c5e8f0379998b4 0.3s
-> => transferring dockerfile: 615B                                0.0s
-> [internal] load metadata for docker.io/library/python:3.8-alpine 0.4s
-> [internal] load .dockerignore                                   0.0s
-> => transferring context: 2B                                       0.0s

```

Figure 1. Creació imatge Docker

Verifiquem que la imatge de Docker s'ha creat llistant les imatges de Docker que coneix el nostre client de Docker, mitjançant l'ordre següent:

```

voclabs:~/environment/containers $ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
api-example v1 d8ab37bd49da About a minute ago 281MB
showtime v1 e00401fcf493 27 minutes ago 128MB

```

Figure 2. Verificació imatge

Execució del contenidor en primer pla: Per iniciar el contenidor manualment i veure la sortida directament al terminal:

```
1 docker run --name api-example -it api-example:v1
```

A continuació mostrem el resultat de l'execució, on es veuen les 3 notícies:

```

voclabs:~/environment/containers $ docker run --name api-example api-example:v1
/usr/local/lib/python3.8/site-packages/fuzzywuzzy/fuzz.py:11: UserWarning: Using
slow pure-python SequenceMatcher. Install python-Levenshtein to remove this w
arnings.warn('Using slow pure-python SequenceMatcher. Install python-Levenshtein
to remove this warning')
Title : Pro-European centrist beats right-wing nationalist in Romanian vote -
The Washington Post
Link : https://news.google.com/rss/articles/CBMilgFBVW95cUXQUGtLVWJ3SlpteERJWD
duV08zBjIra3dDejha2tWk5QOHl0SmhUCExoUG13akpbGmWzhiUnlGUGNlce3JhVexTU0Zmb1RrR
DRYUllSvXhYz6FNS2KxbHV4Qkc0UmN5b192cxHfrko5V0f1fMTBMTG5QZkzhby1TnNjNmVYUw0X05W
azV1Y3JlYnYzSxc?oc=5

Title : 5 years since Brexit, are Britain and the EU getting back together? -
NPR
Link : https://news.google.com/rss/articles/CBMieEFVX3l3lTFBfBmhfMTmwOVN1p4Zn
dpcVWOTv1vbtHMSG55RFZjAhBIREtqNDFURzBTvZNMdxdlYz1fR19IQzFybWdyZmZmQmQVVGUGhTc3NpS
HVP0XrtQ3lralJWUFlvaZn6cW5aRjhm1RqbhlQZ3Utt1BadGwxaA?oc=5

Title : Israel begins extensive Gaza ground operation after intense airstrikes
kill more than 100 overnight - CNN
Link : https://news.google.com/rss/articles/CBMiAFBVW95cUXQUGtLVWJ3SlpteERJWD
duV08zBjIra3dDejha2tWk5QOHl0SmhUCExoUG13akpbGmWzhiUnlGUGNlce3JhVexTU0Zmb1RrR
DRYUllSvXhYz6FNS2KxbHV4Qkc0UmN5b192cxHfrko5V0f1fMTBMTG5QZkzhby1TnNjNmVYUw0X05W
azV1Y3JlYnYzSxc?oc=5

```

Figure 3. Execució del contenidor

Execució en segon pla amb neteja automàtica: Si volguéssim executar el contenidor en segon pla (-d) i eliminar-lo automàticament quan finalitza (-rm), fem servir:

```
1 docker run --rm -d --name api-example api-example:v1
```

Per consultar els registres del contenidor en temps real i fer-ne un seguiment actiu hem de fer:

```
1 docker logs -f api-example
```

Llavors els resultats per pantalla són els mateixos que a la figura 3.

5. Aturada del contenidor

Tornem al cas d'execució en primer pla. Per aturar (Figure 4) i eliminar (Figure 5) el contenidor fem:

```
voclabs:~/environment/containers $ docker stop api-example
api-example
```

Figure 4. Stop container

```
voclabs:~/environment/containers $ docker rm api-example
api-example
```

Figure 5. Remove container

Finalment, comprovem que el contenidor ja no està al nostre sistema amb la comanda:

```
voclabs:~/environment/containers $ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
voclabs:~/environment/containers $
```

Figure 6. Comprovació dockers

Problemes i Solucions

Durant aquesta pràctica no hem trobat gaires inconvenients:

Problema: Error amb `wget` per descarregar `code.zip` (no es copiava el guió de `us-west2`). **Solució:**

Vam corregir la URL a `us-west-2` i vam tornar a executar la comanda.

Problema: Contenidors aturats romanien a la llista amb `docker ps -a`. **Solució:**

Vam utilitzar `docker rm api-example` després de `docker stop`.

Conclusió i beneficis

En aquesta pràctica hem après a dissenyar i utilitzar un `Dockerfile` per crear una imatge lleugera, instal·lar-hi les dependències necessàries i empaquetar-hi un script Python. Hem practicat les comandes bàsiques de Docker per construir la imatge, executar el contenidor tant en primer pla com en segon pla, i monitoritzar la sortida. Finalment, hem revisat la forma d'aturar i esborrar contenidors de manera eficient, consolidant els conceptes de capes i punts d'entrada en l'ecosistema de contenidors Docker.

Pel que fa als beneficis d'utilitzar Docker, veiem que ens permet empaquetar aplicacions i totes les seves dependències en contenidors lleugers, assegurant consistència entre entorns de desenvolupament, proves i producció, facilitant la escalabilitat i la portabilitat. AWS Cloud9, al seu torn, ens ofereix un entorn de desenvolupament integrat al núvol amb accés

directe a recursos AWS, col·laboració en temps real i configuració automàtica d'eines com Docker, eliminant la necessitat d'instal·lar localment i accelerant la posada en marxa dels projectes. Juntament, Docker i Cloud9 optimitzen el flux de treball, redueixen el temps de configuració i ens garantitzen entorns reproducibles.